LAPORAN PRAKTIKUM BASIS DATA
MODUL 10: DBMS DAN APLIKASI BASIS DATA

Nama	:	KHANSA QAIS FERNANDA
NIM	:	264311019
Kelas	:	TRPL 1A
Kelompok	:	KELOMPOK MAC OS
Anggota & Skor	:	1. Khansa Qais Fernanda (264311019) - Skor: 4 (Sangat Baik)
		2. Jelita Palupi (264311017) - Skor: 4 (Sangat Baik)
		3. Zahro Choirun Latifah (264311030) - Skor: 4 (Sangat Baik)
Studi Kasus	:	Implementasi Basis Data Penjualan Sederhana (praktikum_modul10)
DBMS	:	MySQL Community Server 8.4.3 (InnoDB) via phpMyAdmin / MySQL CLI

10.2 SOAL PRE-TEST

1. Jelaskan apa yang dimaksud dengan DBMS dan apa perbedaannya dengan aplikasi pengelola basis data seperti phpMyAdmin atau MySQL Workbench! [Bobot: 30]

JAWAB:

- Pengertian DBMS: Database Management System (DBMS) adalah perangkat lunak yang digunakan untuk membuat, menyimpan, mengakses, mengelola, dan memelihara basis data (mengelola penyimpanan data, memproses SQL, mengatur transaksi, hak akses, dan integritas data). Contohnya: MySQL, MariaDB, PostgreSQL.

- Perbedaan dengan Aplikasi Pengelola: DBMS (Database Server seperti MySQL) adalah mesin inti yang menyimpan dan memproses basis data di belakang layar, sedangkan aplikasi pengelola basis data (Database Management Client seperti phpMyAdmin, MySQL Workbench, DBeaver) adalah antarmuka yang menghubungkan pengguna dengan DBMS untuk memudahkan pengelolaan.

2. Apa fungsi Primary Key dan Foreign Key dalam menjaga integritas data pada basis data relasional? [Bobot: 30]

JAWAB:

- Fungsi Primary Key (PK): Digunakan sebagai identitas unik untuk setiap baris data pada tabel (misalnya id_pelanggan, id_produk, id_pesanan) dan tidak boleh kosong (NOT NULL), sehingga mencegah duplikasi data induk.

- Fungsi Foreign Key (FK): Menjaga hubungan dan integritas referensial antartabel dengan memastikan kolom kunci tamu pada tabel anak selalu merujuk pada Primary Key yang sah di tabel induk. FK mencegah transaksi merujuk ke pelanggan/produk yang tidak terdaftar serta mencegah penghapusan data induk yang masih digunakan di tabel transaksi (ON DELETE RESTRICT).

3. Mengapa penggunaan indeks dapat membantu meningkatkan performa query, dan apakah semakin banyak indeks selalu membuat basis data semakin cepat? Jelaskan! [Bobot: 40]

JAWAB:

- Alasan Indeks Meningkatkan Performa Query: Index adalah struktur pencarian tambahan (B-Tree) pada kolom tertentu yang membantu DBMS menemukan data dengan cepat tanpa harus memeriksa seluruh baris tabel satu per satu (Full Table Scan).

- Apakah Semakin Banyak Indeks Selalu Lebih Cepat? Tidak. Jika data masih sangat sedikit, optimizer dapat memilih melakukan table scan biasa karena lebih efisien. Selain itu, terlalu banyak indeks justru memperlambat operasi INSERT, UPDATE, dan DELETE karena setiap perubahan data mengharuskan

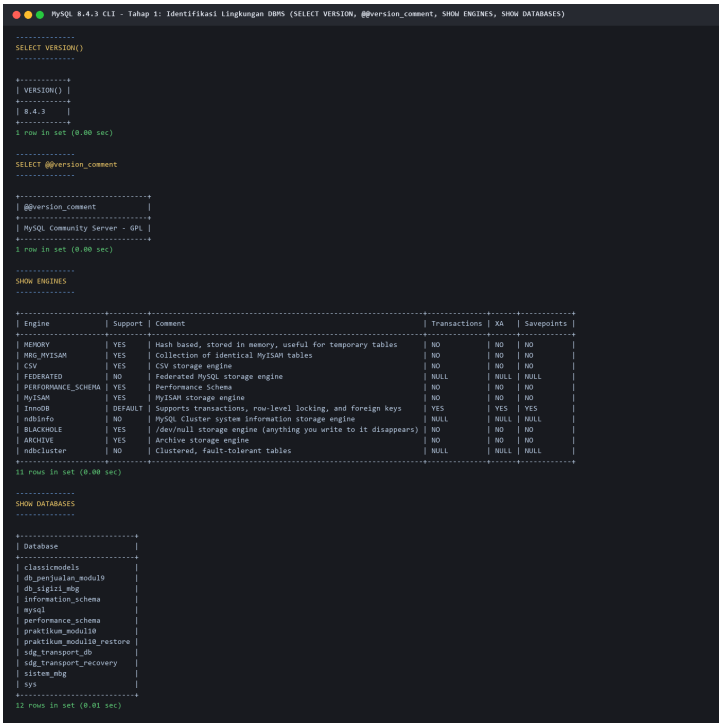
DBMS ikut memperbarui semua indeks yang ada.

10.3 TUGAS / UNJUK KERJA (PRAKTIKUM & PRESENTASI)

1. Tahap 1 - Eksplorasi DBMS dan Arsitektur Sistem [Skor: 25]

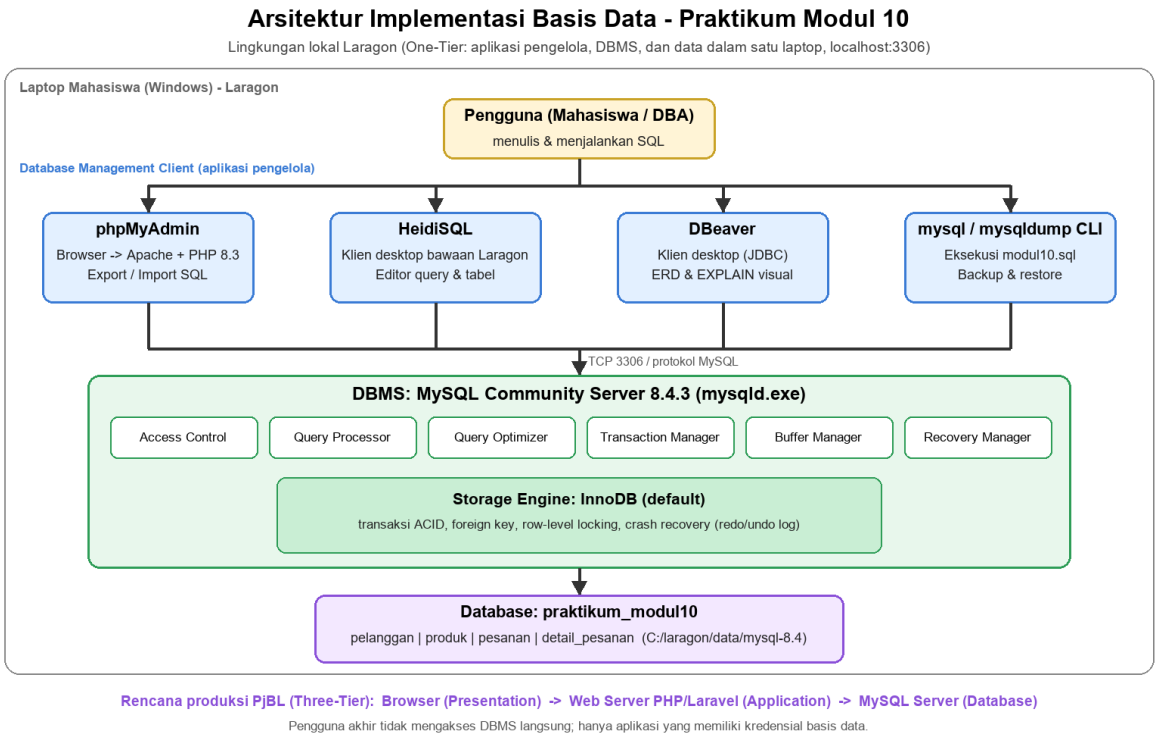
JAWAB:

```
SELECT VERSION();
SELECT @@version_comment;
SHOW ENGINES;
SHOW DATABASES;
```



Gambar 1. Hasil Identifikasi Lingkungan DBMS (SELECT VERSION, @@version_comment, SHOW ENGINES, SHOW DATABASES)

1. Identifikasi DBMS: MySQL Community Server - GPL versi 8.4.3 dengan default storage engine InnoDB.
2. Aplikasi Pengelola: phpMyAdmin & MySQL Command Line (CLI) melalui Laragon.
3. Kelebihan & Keterbatasan: phpMyAdmin mudah digunakan secara visual melalui browser, namun dibatasi ukuran unggah saat memproses jutaan data; diatasi dengan MySQL CLI (mysql & mysqldump) yang cepat.
4. Alasan Pemilihan & Arsitektur: Gratis, kompatibel penuh, serta mendukung transaksi ACID dan Foreign Key. Praktikum lokal menggunakan One-Tier Architecture, sedangkan proyek web PjBL menggunakan Three-Tier Architecture.



Gambar 2. Diagram Arsitektur Implementasi Sistem Basis Data (arsitektur_db.png)

2. Tahap 2 - Implementasi Basis Data [Skor: 30 - Bagian A]

JAWAB:

```
CREATE DATABASE praktikum_modul10;
USE praktikum_modul10;

CREATE TABLE pelanggan (
   id_pelanggan INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
   nama_pelanggan VARCHAR(100) NOT NULL,
   email VARCHAR(120) NOT NULL UNIQUE,
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE produk (
   id_produk INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
   nama_produk VARCHAR(100) NOT NULL,
   harga_produk DECIMAL(12,2) NOT NULL,
   stok INT UNSIGNED NOT NULL DEFAULT 0
) ENGINE=InnoDB;

CREATE TABLE pesanan (
   id_pesanan INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
   id_pelanggan INT UNSIGNED NOT NULL,
   tanggal DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
   status_pesanan ENUM('baru', 'diproses', 'selesai', 'batal') NOT NULL DEFAULT 'baru',
   CONSTRAINT fk_pesanan_pelanggan
       FOREIGN KEY (id_pelanggan) REFERENCES pelanggan(id_pelanggan)
       ON UPDATE CASCADE
       ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE detail_pesanan (
   id_pesanan INT UNSIGNED NOT NULL,
```

```
   id_produk INT UNSIGNED NOT NULL,
   jumlah INT UNSIGNED NOT NULL,
   harga_saat_transaksi DECIMAL(12,2) NOT NULL,
   PRIMARY KEY (id_pesanan, id_produk),
   CONSTRAINT fk_detail_pesanan
       FOREIGN KEY (id_pesanan) REFERENCES pesanan(id_pesanan)
       ON UPDATE CASCADE
       ON DELETE RESTRICT,
   CONSTRAINT fk_detail_produk
       FOREIGN KEY (id_produk) REFERENCES produk(id_produk)
       ON UPDATE CASCADE
       ON DELETE RESTRICT
) ENGINE=InnoDB;

SHOW TABLES;
SHOW TABLE STATUS;
SHOW CREATE TABLE pelanggan;
SHOW CREATE TABLE pesanan;
SHOW CREATE TABLE detail_pesanan;
```



Gambar 3. Hasil Pemeriksaan Tabel dan Relasi (SHOW TABLES, SHOW TABLE STATUS, SHOW CREATE TABLE)

3. Tahap 3 - Praktik Operasi Database (CRUD) & Integritas Referensial [Skor: 30 - Bagian B]

JAWAB:

```
INSERT INTO pelanggan (id_pelanggan, nama_pelanggan, email) VALUES
(1, 'Budi', 'budi@example.test'),
(2, 'Siti', 'siti@example.test');

INSERT INTO produk (id_produk, nama_produk, harga_produk, stok) VALUES
(10, 'Keyboard', 200000, 15),
```

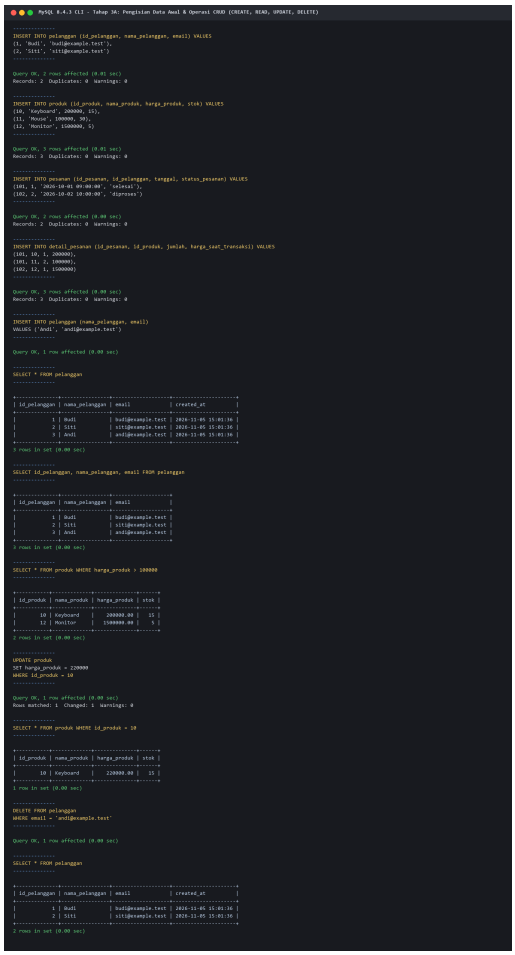
```
(11, 'Mouse', 100000, 30),
(12, 'Monitor', 1500000, 5);

INSERT INTO pesanan (id_pesanan, id_pelanggan, tanggal, status_pesanan) VALUES
(101, 1, '2026-10-01 09:00:00', 'selesai'),
(102, 2, '2026-10-02 10:00:00', 'diproses');

INSERT INTO detail_pesanan (id_pesanan, id_produk, jumlah, harga_saat_transaksi) VALUES
(101, 10, 1, 200000),
(101, 11, 2, 100000),
(102, 12, 1, 1500000);

INSERT INTO pelanggan (nama_pelanggan, email)
VALUES ('Andi', 'andi@example.test');
SELECT * FROM pelanggan;
SELECT id_pelanggan, nama_pelanggan, email FROM pelanggan;
SELECT * FROM produk WHERE harga_produk > 100000;
UPDATE produk SET harga_produk = 220000 WHERE id_produk = 10;
SELECT * FROM produk WHERE id_produk = 10;
DELETE FROM pelanggan WHERE email = 'andi@example.test';
SELECT * FROM pelanggan;

INSERT INTO pesanan (id_pesanan, id_pelanggan, tanggal)
VALUES (105, 999, '2026-10-05 10:00:00');
DELETE FROM pelanggan WHERE id_pelanggan = 1;
```



Gambar 4. Hasil Pengisian Data Awal dan Operasi CRUD (CREATE, READ, UPDATE, DELETE)



Gambar 5. Bukti Pengujian Integritas Referensial (ERROR 1452 & ERROR 1451)

4. Tahap 4 - Pengujian Transaksi Database (COMMIT & ROLLBACK) [Skor: 25 - Bagian A]

JAWAB:

```
START TRANSACTION;
INSERT INTO pesanan (id_pesanan, id_pelanggan, tanggal, status_pesanan)
VALUES (103, 1, '2026-10-05 09:00:00', 'baru');
INSERT INTO detail_pesanan (id_pesanan, id_produk, jumlah, harga_saat_transaksi)
VALUES (103, 11, 2, 100000);
COMMIT;
SELECT * FROM pesanan WHERE id_pesanan = 103;
SELECT * FROM detail_pesanan WHERE id_pesanan = 103;

START TRANSACTION;
INSERT INTO pesanan (id_pesanan, id_pelanggan, tanggal, status_pesanan)
VALUES (104, 2, '2026-10-06 10:00:00', 'baru');
INSERT INTO detail_pesanan (id_pesanan, id_produk, jumlah, harga_saat_transaksi)
VALUES (104, 10, 1, 200000);
ROLLBACK;
SELECT * FROM pesanan WHERE id_pesanan = 104;
```



Gambar 6. Hasil Pengujian COMMIT (Pesanan 103 Tersimpan) dan ROLLBACK (Pesanan 104 Tidak Tersimpan)

- Perbedaan COMMIT & ROLLBACK: COMMIT menyimpan perubahan transaksi secara permanen (pesanan 103 tersimpan), sedangkan ROLLBACK membatalkan seluruh perubahan sejak START TRANSACTION (pesanan 104 tidak tersimpan / Empty set).
- Pentingnya Atomicity & InnoDB: Menjamin penyimpanan tabel pesanan dan detail_pesanan berjalan sebagai satu kesatuan utuh berkat dukungan transaksi ACID pada engine InnoDB.

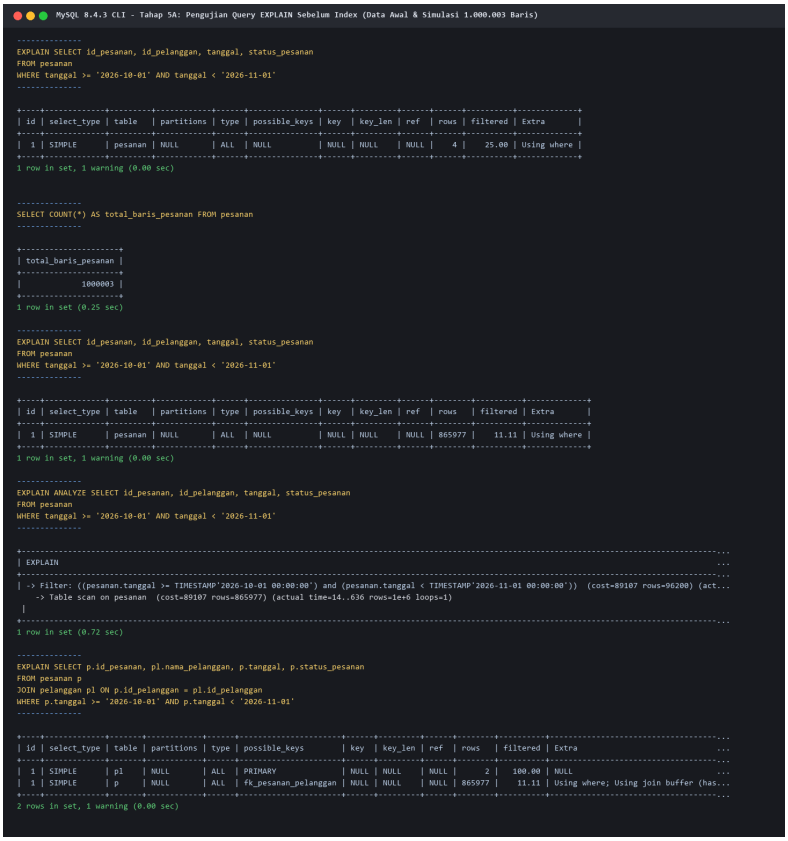
5. Tahap 5 - Analisis Performa Database (EXPLAIN & Index) [Skor: 25 - Bagian B]

JAWAB:

```
EXPLAIN SELECT id_pesanan, id_pelanggan, tanggal, status_pesanan
FROM pesanan
WHERE tanggal >= '2026-10-01' AND tanggal < '2026-11-01';

CREATE INDEX idx_pesanan_tanggal ON pesanan(tanggal);
SHOW INDEX FROM pesanan;

EXPLAIN SELECT id_pesanan, id_pelanggan, tanggal, status_pesanan
FROM pesanan
WHERE tanggal >= '2026-10-01' AND tanggal < '2026-11-01';
```



Gambar 7. Hasil EXPLAIN Sebelum Penambahan Index (type = ALL / Full Table Scan)



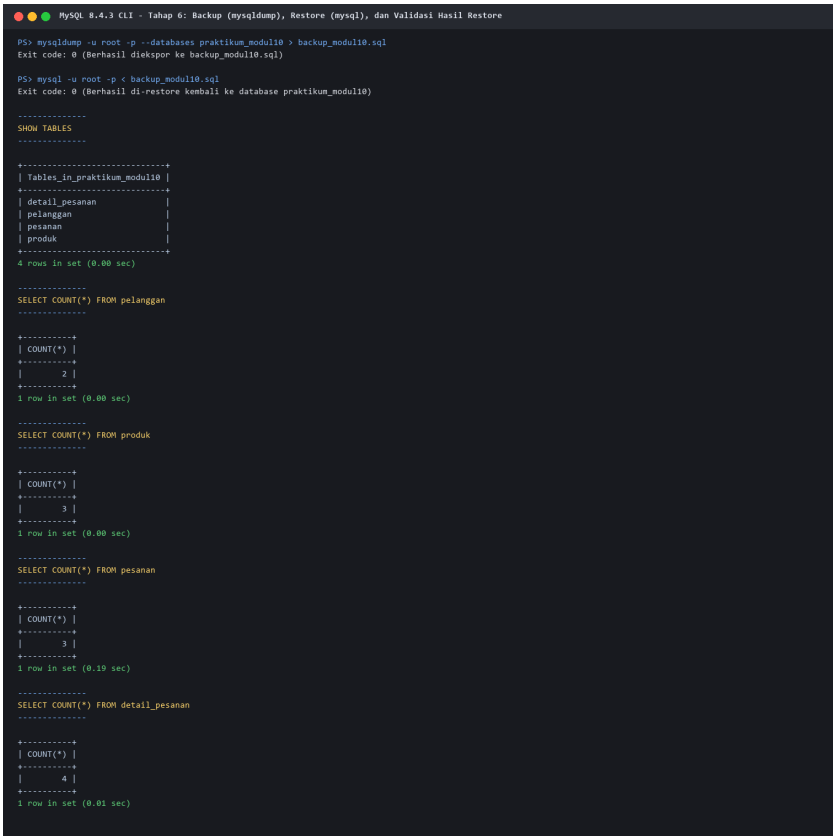
Gambar 8. Pembuatan Index idx_pesanan_tanggal dan Hasil EXPLAIN Sesudah Index (type = range)

6. Tahap 6 - Pemeliharaan Basis Data (Backup, Restore & Git) [Skor: 20]

JAWAB:

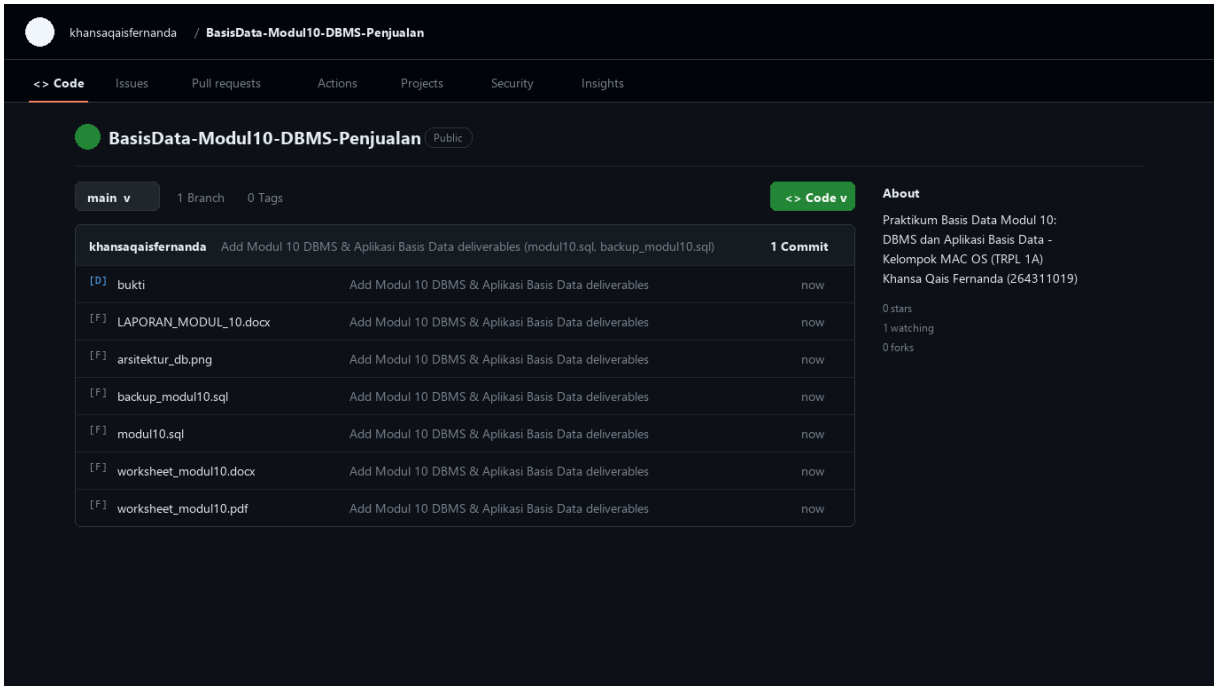
```
mysqldump -u root -p --databases praktikum_modul10 > backup_modul10.sql
```

```
mysql -u root -p < backup_modul10.sql
USE praktikum_modul10;
SHOW TABLES;
SELECT COUNT(*) FROM pelanggan;
SELECT COUNT(*) FROM produk;
SELECT COUNT(*) FROM pesanan;
SELECT COUNT(*) FROM detail_pesanan;
```



Gambar 9. Bukti Backup, Restore, dan Validasi Hasil Restore (2 pelanggan, 3 produk, 3 pesanan, 4 detail_pesanan)

Link GitHub: https://github.com/khansaqaisfernanda/BasisData-Modul10-DBMS-Penjualan



Gambar 10. Bukti Penyimpanan Berkas modul10.sql dan backup_modul10.sql pada Repositori GitHub

---

10.4 LEMBAR KERJA (POST-TEST)

1. Instruksi 1 (Bobot 25): Identifikasi DBMS, versi, storage engine, aplikasi pengelola, diagram arsitektur, dan alasan pemilihan!

JAWAB: Menggunakan MySQL Community Server 8.4.3 dengan storage engine InnoDB, dikelola dengan phpMyAdmin dan MySQL CLI pada lingkungan Laragon (One-Tier Architecture untuk praktikum lokal, Three-Tier Architecture untuk web PjBL). Dipilih karena gratis, kompatibel, dan mendukung transaksi ACID serta Foreign Key.

2. Instruksi 2 (Bobot 30): Script SQL pembuatan tabel beserta PK, FK, hasil pengujian CRUD, dan penjelasan constraint!

JAWAB: Terlampir pada Tahap 2 & 3 (Gambar 3, 4, 5). Primary Key menjaga keunikan baris, UNIQUE mencegah email ganda, dan Foreign Key (ON DELETE RESTRICT) menolak transaksi untuk pelanggan yang belum terdaftar (id_pelanggan = 999 ditolak dengan ERROR 1452) serta menolak penghapusan pelanggan yang masih memiliki transaksi (id_pelanggan = 1 ditolak dengan ERROR 1451).

3. Instruksi 3 (Bobot 25): Hasil EXPLAIN sebelum dan sesudah pembuatan indeks serta analisis performanya!

JAWAB: Terlampir pada Gambar 7 & 8. Penambahan indeks idx_pesanan_tanggal mengubah rencana eksekusi dari Full Table Scan (type = ALL, key = NULL, rows = 865977, waktu 717 ms) menjadi Index Range Scan (type = range, key = idx_pesanan_tanggal, rows = 3, waktu 0,034 ms).

4. Instruksi 4 (Bobot 20): Bukti backup dan restore serta alasan pentingnya pemeliharaan dalam DBMS!

JAWAB: Terlampir pada Gambar 9. Hasil validasi setelah restore menunjukkan jumlah data tetap utuh (pelanggan = 2, produk = 3, pesanan = 3, detail_pesanan = 4). Pemeliharaan dan backup berkala penting untuk menjaga ketersediaan data dan memulihkan basis data saat terjadi kegagalan sistem.

10.5 KESIMPULAN

Penggunaan storage engine InnoDB beserta constraint PK, FK (ON DELETE RESTRICT), UNIQUE, dan NOT NULL menjaga integritas data secara ketat, transaksi COMMIT dan ROLLBACK menjamin konsistensi operasi multi-tabel, indeks idx_pesanan_tanggal mempercepat pencarian berdasarkan EXPLAIN dari Full Table Scan menjadi Index Range Scan, dan backup/restore menjaga ketersediaan data.